In [2]:

```
# === Prepare Data ===
# Step 1/5 import packages
import pandas as pd
import matplotlib.pyplot as plt
import holidays
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from xgboost import XGBRegressor
import numpy as np
################################################################################
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce')   # parse with offsets
df = df.set_index('time')
df = df.dropna()

def Predicting_customers(df):
    df['year'] = df.index.year
    df['month'] = df.index.month
    df['day'] = df.index.day
    df['day_of_week'] = df.index.dayofweek

    df = df.copy()
    # yesterday's customers
    df["previous_day_customers"] = df["customers"].shift(1).fillna(0)
    df = df.copy()
        # rolling mean up to yesterday (no leakage)
    df["rolling_mean"] = df["customers"].shift(1).rolling(window=5, min_periods=
    df["rolling_mean"] = df["rolling_mean"].fillna(df["rolling_mean"].mean())
    df = df.copy()
        # exponential smoothing up to yesterday
    df["exponential_smoothing"] = df["customers"].shift(1).ewm(span=5, adjust=Fa
    df["exponential_smoothing"] = df["exponential_smoothing"].fillna(df["exponen
    df = df.copy()
        # rolling autocorrelation up to yesterday
    roll_autocorr = (
        df["customers"].shift(1)  # avoid peeking at today
        .rolling(window=14, min_periods=max(1 + 1, 3))
        .apply(lambda x: x.autocorr(lag=1), raw=False)
        )
    df["autocorrelation"] = roll_autocorr.fillna(0)  # safer default than mean
    df = df.copy()
    if not isinstance(df.index, pd.DatetimeIndex):
        raise ValueError("DataFrame index must be a DatetimeIndex.")
    fi_holidays = holidays.Finland(years=range(2019, 2025))
    holiday_idx = pd.DatetimeIndex(pd.to_datetime(list(fi_holidays.keys())))
    d = df.index.normalize()

    df["is_holiday"] = d.isin(holiday_idx).astype(int)
    df["day_before_holiday"] = (d + pd.Timedelta(days=1)).isin(holiday_idx).asty
    df["day_after_holiday"] = (d - pd.Timedelta(days=1)).isin(holiday_idx).astyp
    df = df.dropna()

    target = "customers"
    features = [col for col in df.columns if col != target]

    X = df[features]

    y = df[target]
```

```
    # === Train/Test Split (time-aware, no shuffle) ===
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, shuffle=False
    )

    # === Model ===
    model = XGBRegressor(
        n_estimators=300,
        learning_rate=0.1,
        max_depth=5,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42
    )

    # === Train ===
    model.fit(X_train, y_train)

    # === Predict ===
    y_pred = model.predict(X_test)

    # === Evaluate ===
    mse  = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae  = mean_absolute_error(y_test, y_pred)
    r2   = r2_score(y_test, y_pred)

    print(f"MSE:  {mse:.2f}")
    print(f"RMSE: {rmse:.2f}")
    print(f"MAE:  {mae:.2f}")
    print(f"R²:   {r2:.2f}")

Predicting_customers(df)
```

```
MSE:  14190.52
RMSE: 119.12
MAE:  96.64
R²:   0.75
```

Null and Zeros in data

In [3]:

```
# Null values per column
null_counts = df.isnull().sum()

# Zero counts per column (numeric only)
zero_counts = (df == 0).sum(numeric_only=True)

print("=== Null Values ===")
print(null_counts)
print("----------------------------------------------------------------------")
print("\n=== Zero Values ===")
print(zero_counts)
```

```
=== Null Values ===
customers      0
year           0
month          0
day            0
day_of_week    0
dtype: int64
------------------------------------------------------------------------

=== Zero Values ===
customers        0
year             0
month            0
day              0
day_of_week    283
dtype: int64
```

In [4]:

```python
import pandas as pd

# Ensure time is datetime index
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce')
df = df.set_index('time')

# Group by year and month
report = {}

for year, df_y in df.groupby(df.index.year):
    # Highest and lowest days
    best_day = df_y['customers'].idxmax(), df_y['customers'].max()
    worst_day = df_y['customers'].idxmin(), df_y['customers'].min()

    # Monthly averages
    monthly_avg = df_y['customers'].resample('M').mean()
    best_month = monthly_avg.idxmax().strftime("%Y-%m"), monthly_avg.max()
    worst_month = monthly_avg.idxmin().strftime("%Y-%m"), monthly_avg.min()

    report[year] = {
        "best_day": best_day,
        "worst_day": worst_day,
        "best_month": best_month,
        "worst_month": worst_month
    }

# Convert to DataFrame for easier display
yearly_report = pd.DataFrame(report).T

print("=== Year-wise Report ===")
print(yearly_report)

# Final comparison across years
final_report = pd.DataFrame({
    "avg_customers": df.groupby(df.index.year)["customers"].mean(),
    "max_customers": df.groupby(df.index.year)["customers"].max(),
    "min_customers": df.groupby(df.index.year)["customers"].min()
})

print("\n=== Comparison Across Years ===")
print(final_report)
```

```
=== Year-wise Report ===
                         best_day                 worst_day  \
2019   (2019-12-02 00:00:00, 992)  (2019-01-24 00:00:00, 3)
2020   (2020-01-22 00:00:00, 883)  (2020-01-05 00:00:00, 3)
2021   (2021-11-16 00:00:00, 774)  (2021-01-05 00:00:00, 3)
2022  (2022-10-25 00:00:00, 1024)  (2022-01-03 00:00:00, 3)
2023  (2023-09-26 00:00:00, 1130)  (2023-01-05 00:00:00, 3)
2024   (2024-02-06 00:00:00, 973)  (2024-03-28 00:00:00, 3)

                        best_month                    worst_month
2019             (2019-11, 749.65)                (2019-01, 5.125)
2020             (2020-02, 633.75)   (2020-05, 5.142857142857143)
2021  (2021-11, 610.6363636363636)  (2021-07, 149.42857142857142)
2022  (2022-11, 805.4545454545455)  (2022-07, 252.23809523809524)
2023             (2023-09, 727.65)  (2023-07, 185.54545454545453)
2024  (2024-02, 660.6190476190476)                (2024-06, 225.2)

=== Comparison Across Years ===
      avg_customers  max_customers  min_customers
time
2019     318.211382            992              3
2020     319.625954            883              3
2021     336.376923            774              3
2022     516.634615           1024              3
2023     535.528736           1130              3
2024     499.899225            973              3
```

```
/run/nvme/job_5101626/tmp/ipykernel_1059644/3892203093.py:17: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  monthly_avg = df_y['customers'].resample('M').mean()
/run/nvme/job_5101626/tmp/ipykernel_1059644/3892203093.py:17: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  monthly_avg = df_y['customers'].resample('M').mean()
/run/nvme/job_5101626/tmp/ipykernel_1059644/3892203093.py:17: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  monthly_avg = df_y['customers'].resample('M').mean()
/run/nvme/job_5101626/tmp/ipykernel_1059644/3892203093.py:17: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  monthly_avg = df_y['customers'].resample('M').mean()
/run/nvme/job_5101626/tmp/ipykernel_1059644/3892203093.py:17: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  monthly_avg = df_y['customers'].resample('M').mean()
/run/nvme/job_5101626/tmp/ipykernel_1059644/3892203093.py:17: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  monthly_avg = df_y['customers'].resample('M').mean()
```

In [5]:

```python
import matplotlib.pyplot as plt

# Ensure datetime index
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce')
df = df.set_index('time')

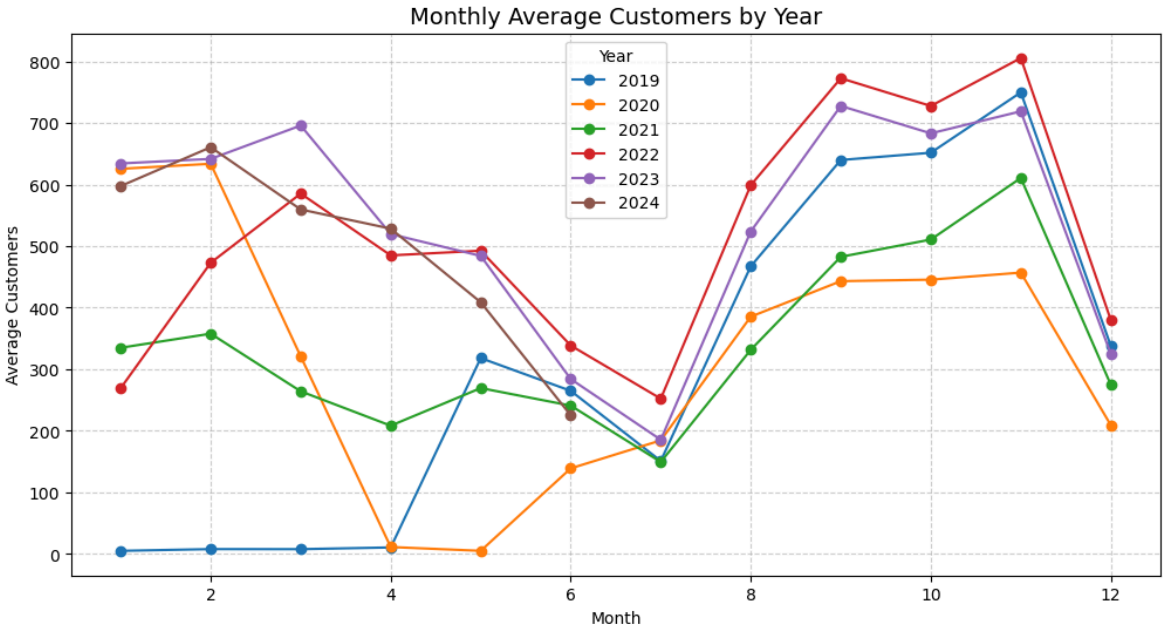
# Compute monthly averages
monthly = df['customers'].resample('M').mean()

# Pivot to Year-Month for plotting
monthly_df = monthly.to_frame()
monthly_df['Year'] = monthly_df.index.year
monthly_df['Month'] = monthly_df.index.month
```

```
pivot = monthly_df.pivot_table(
    index='Month', columns='Year', values='customers'
)

# Plot
plt.figure(figsize=(12, 6))
pivot.plot(marker="o", figsize=(12, 6))
plt.title("Monthly Average Customers by Year", fontsize=14)
plt.xlabel("Month")
plt.ylabel("Average Customers")
plt.legend(title="Year")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```

```
/run/nvme/job_5101626/tmp/ipykernel_1059644/1839069933.py:9: FutureWarning: 'M' i
s deprecated and will be removed in a future version, please use 'ME' instead.
  monthly = df['customers'].resample('M').mean()
<Figure size 1200x600 with 0 Axes>
```



In [6]:
```
import pandas as pd

# Load data
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce')
df = df.set_index('time')

# Flag anomalies: anything <= 50 customers
df["is_anomaly"] = (df["customers"] <= 50).astype(int)

# Separate into normal vs anomalies
normal_data = df[df["is_anomaly"] == 0]
anomalies   = df[df["is_anomaly"] == 1]

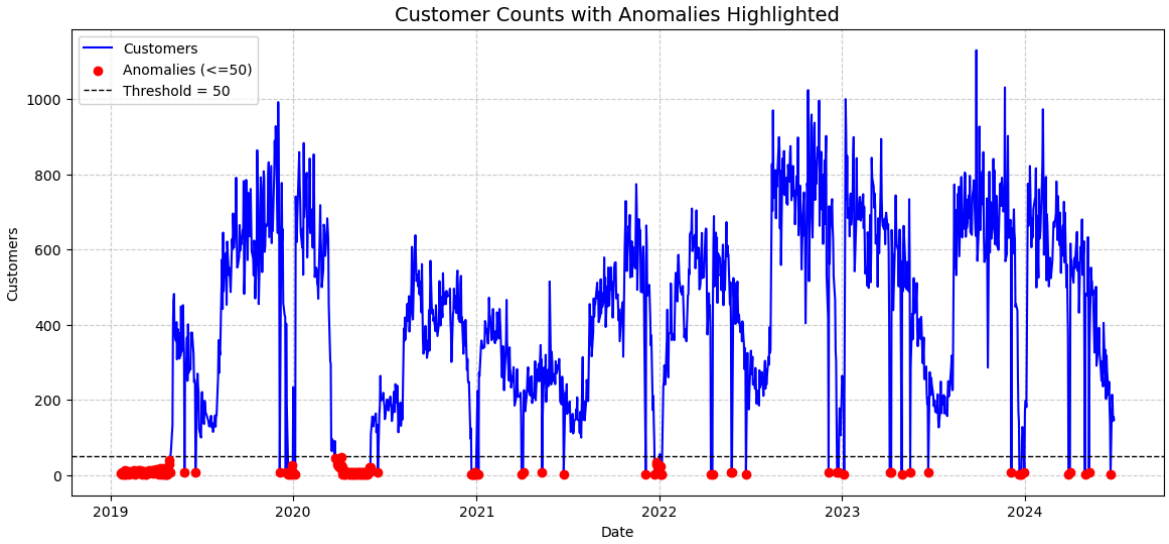
print("Normal rows:", len(normal_data))
print("Anomaly rows:", len(anomalies))
```

```
Normal rows: 1235
Anomaly rows: 183
```

In [7]:
```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(14, 6))
plt.plot(df.index, df["customers"], label="Customers", color="blue")
plt.scatter(anomalies.index, anomalies["customers"], color="red", label="Anomali

plt.axhline(50, color="black", linestyle="--", linewidth=1, label="Threshold = 5
plt.title("Customer Counts with Anomalies Highlighted", fontsize=14)
plt.xlabel("Date")
plt.ylabel("Customers")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```



In [8]:
```
import pandas as pd

# Load data
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce')
df = df.set_index('time')

# Compute mean and std
mean_val = df["customers"].mean()
std_val  = df["customers"].std()

# Define threshold: mean + 3*std (classic anomaly detection rule)
upper_threshold = mean_val + 2 * std_val

# Flag anomalies
df["is_high_anomaly"] = (df["customers"] > upper_threshold).astype(int)

# Separate
high_anomalies = df[df["is_high_anomaly"] == 1]

print(f"Mean customers: {mean_val:.2f}")
print(f"Upper anomaly threshold: {upper_threshold:.2f}")
print(f"High anomalies found: {len(high_anomalies)}")
```

```
Mean customers: 414.71
Upper anomaly threshold: 922.87
High anomalies found: 14
```

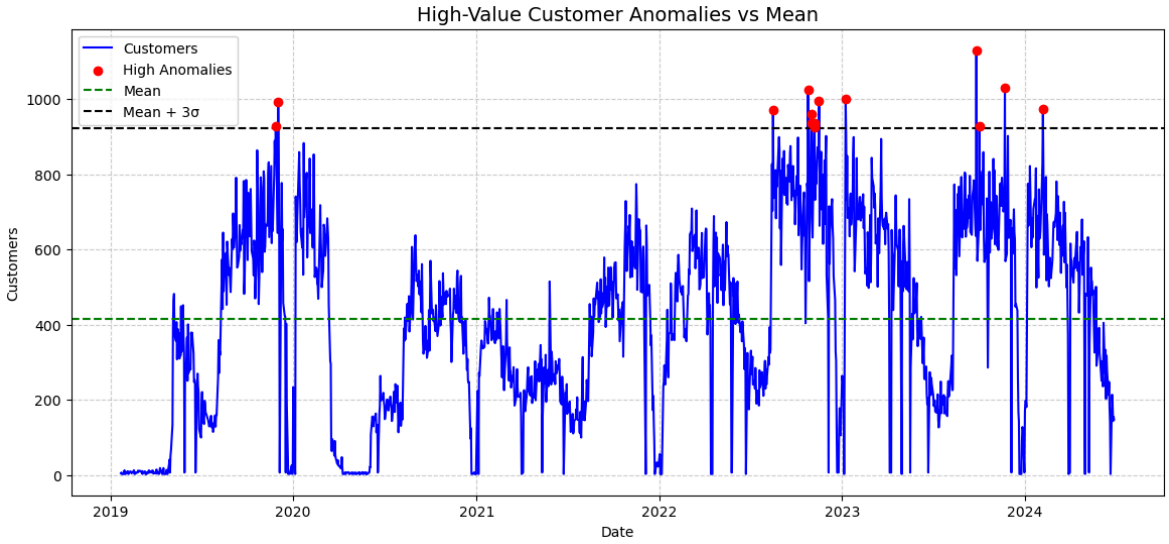
In [9]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 6))
```

```
plt.plot(df.index, df["customers"], label="Customers", color="blue")
plt.scatter(high_anomalies.index, high_anomalies["customers"], color="red", labe

plt.axhline(mean_val, color="green", linestyle="--", label="Mean")
plt.axhline(upper_threshold, color="black", linestyle="--", label="Mean + 3σ")

plt.title("High-Value Customer Anomalies vs Mean", fontsize=14)
plt.xlabel("Date")
plt.ylabel("Customers")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```



In [10]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce')
df = df.set_index('time')

# Mean and std
mean_val = df["customers"].mean()
std_val  = df["customers"].std()
upper_threshold = mean_val + 2 * std_val

# Combined anomaly flag
df["is_anomaly"] = (
    (df["customers"] <= 50) | (df["customers"] > upper_threshold)
).astype(int)

# Split sets
anomalies = df[df["is_anomaly"] == 1]
normal    = df[df["is_anomaly"] == 0]

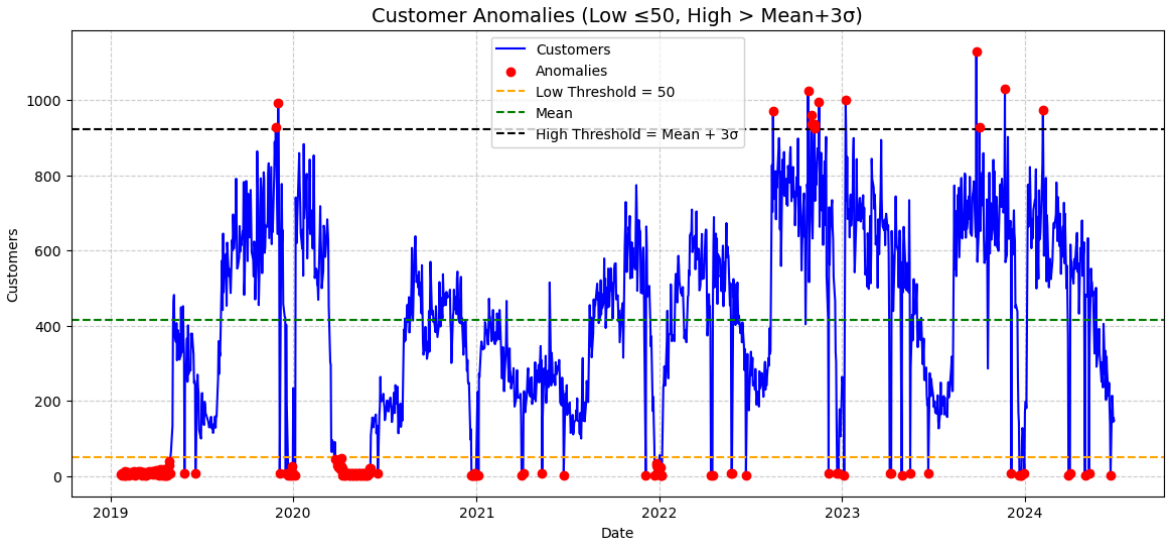
print(f"Mean customers: {mean_val:.2f}")
print(f"Upper threshold (mean + 3σ): {upper_threshold:.2f}")
print(f"Low anomalies (<=50): {sum(df['customers'] <= 50)}")
print(f"High anomalies (> mean+3σ): {sum(df['customers'] > upper_threshold)}")
print(f"Total anomalies: {len(anomalies)}")
```

```
Mean customers: 414.71
Upper threshold (mean + 3σ): 922.87
Low anomalies (<=50): 183
High anomalies (> mean+3σ): 14
Total anomalies: 197
```

In [14]:
```
plt.figure(figsize=(14, 6))
plt.plot(df.index, df["customers"], label="Customers", color="blue")
plt.scatter(anomalies.index, anomalies["customers"], color="red", label="Anomali

plt.axhline(50, color="orange", linestyle="--", label="Low Threshold = 50")
plt.axhline(mean_val, color="green", linestyle="--", label="Mean")
plt.axhline(upper_threshold, color="black", linestyle="--", label="High Threshol

plt.title("Customer Anomalies (Low ≤50, High > Mean+3σ)", fontsize=14)
plt.xlabel("Date")
plt.ylabel("Customers")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.savefig("anomalies.png")
plt.show()
```



In [ ]:
```
```

In [16]:
```
import pandas as pd

# Load data
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce')
df = df.set_index('time')

# Remove low anomalies (customers <= 50)
clean_df = df[df["customers"] > 50]

print(f"Original rows: {len(df)}")
print(f"Clean rows (low anomalies removed): {len(clean_df)}")
print(f"Removed rows: {len(df) - len(clean_df)}")
```

```
Original rows: 1418
Clean rows (low anomalies removed): 1235
Removed rows: 183
```

In [21]:
```
df = df[df["customers"] > 50]
Predicting_customers(df)
```

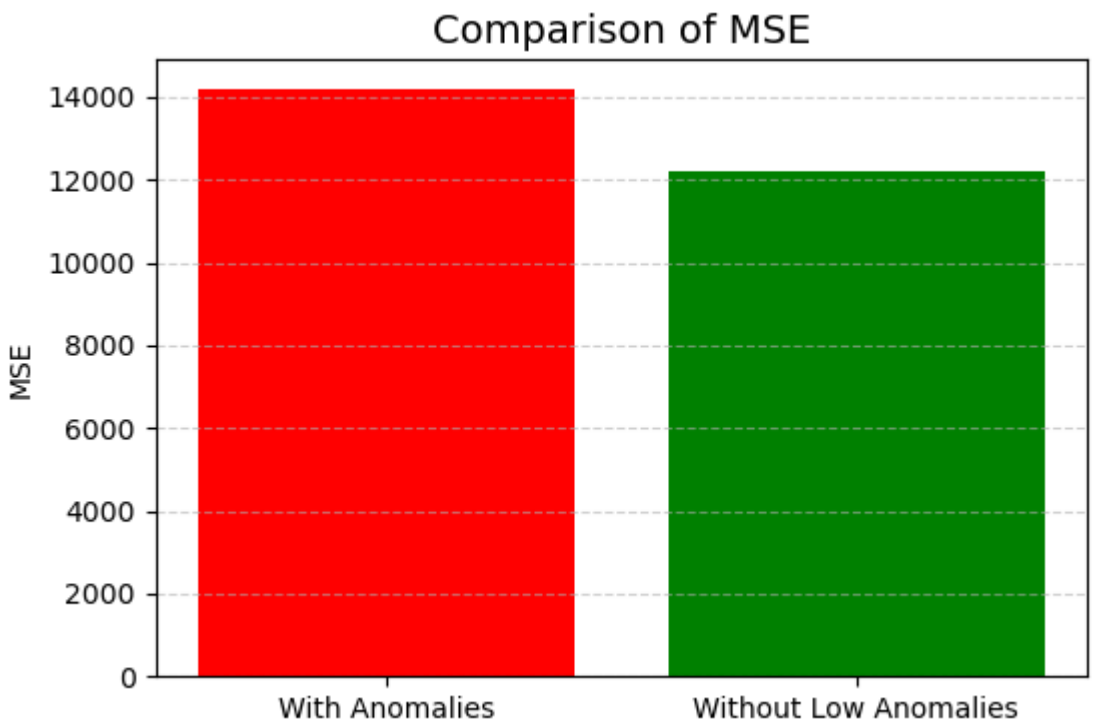
```
MSE:  12196.38
RMSE: 110.44
MAE:  82.92
R²:   0.74
```

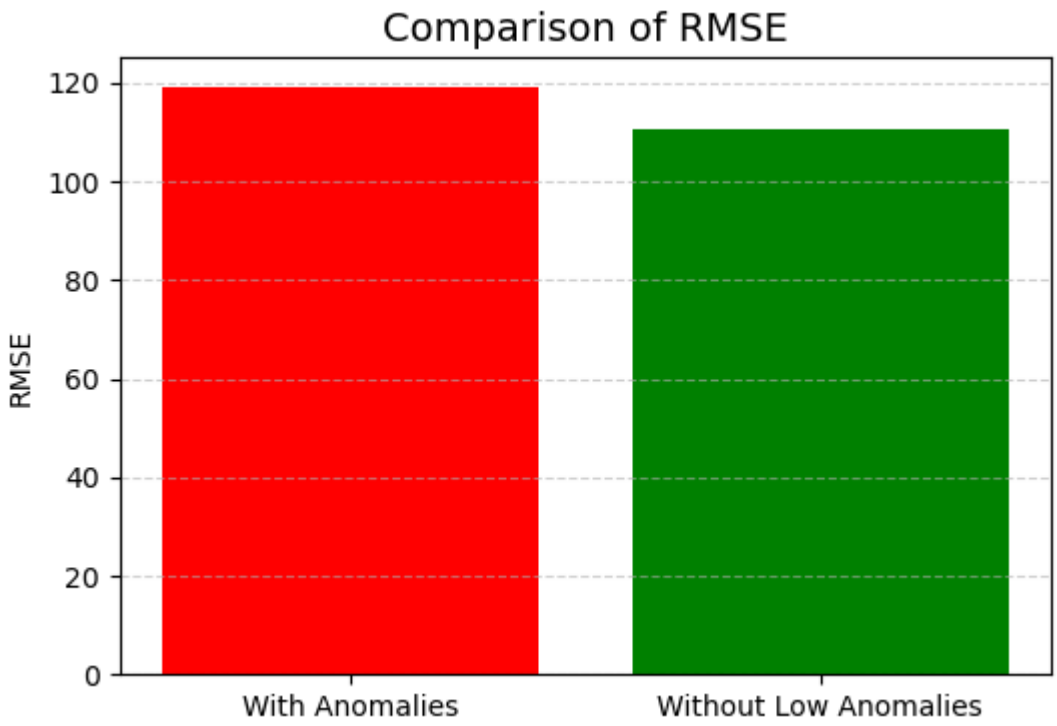
In [23]:
```
import matplotlib.pyplot as plt

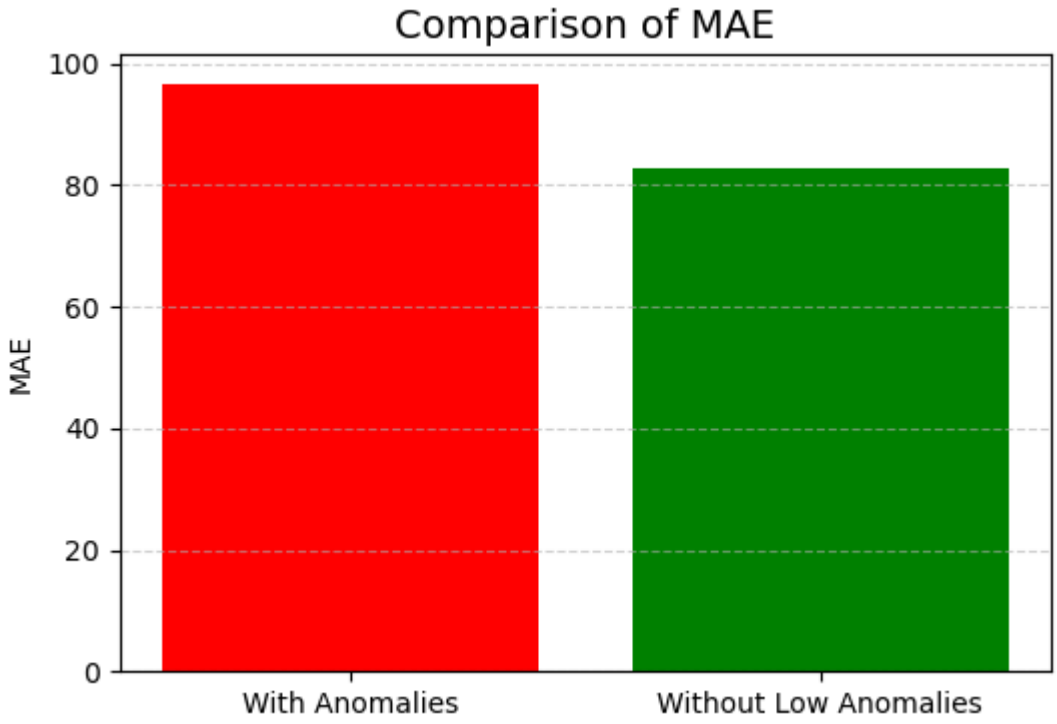
# Values you reported
with_anomalies     = {"MSE": 14190.52, "RMSE": 119.12, "MAE": 96.64, "R²": 0.75}
without_anomalies  = {"MSE": 12196.38, "RMSE": 110.44, "MAE": 82.92, "R²": 0.74}

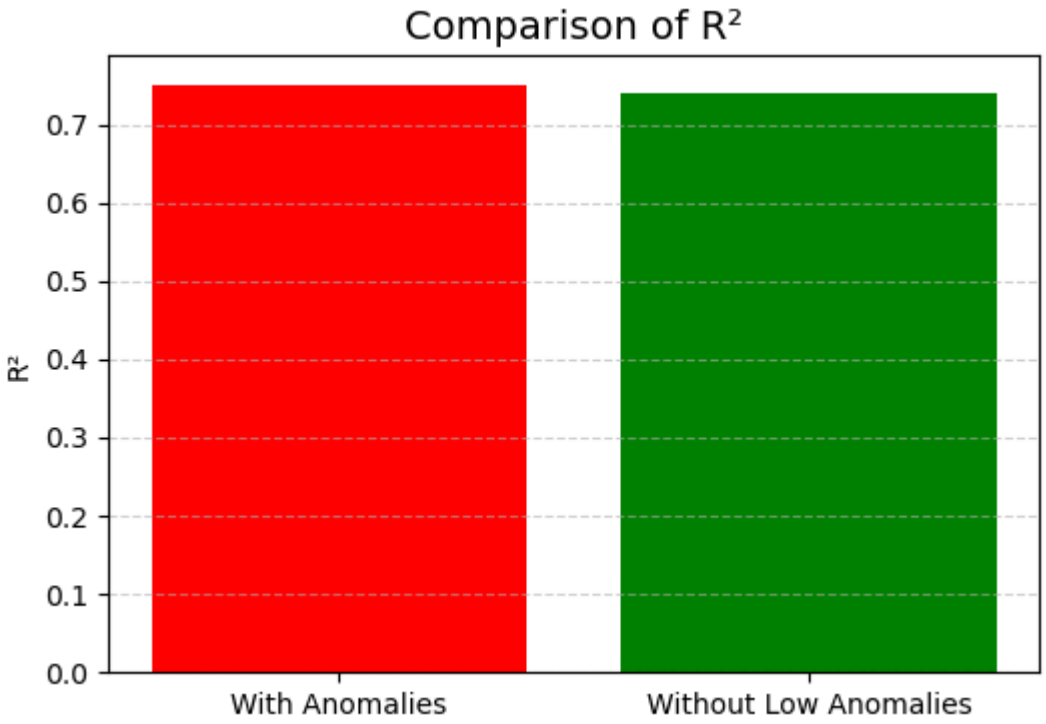
metrics = list(with_anomalies.keys())

# Plot each metric separately
for metric in metrics:
    plt.figure(figsize=(6,4))
    plt.bar(["With Anomalies", "Without Low Anomalies"],
            [with_anomalies[metric], without_anomalies[metric]],
            color=["red", "green"])
    plt.title(f"Comparison of {metric}", fontsize=14)
    plt.ylabel(metric)
    plt.grid(axis="y", linestyle="--", alpha=0.6)
    plt.show()
```




Comparison of RMSE
RMSE
120
100
80
60
40
20
0
With Anomalies
Without Low Anomalies


Comparison of MAE
MAE
100
80
60
40
20
0
With Anomalies
Without Low Anomalies


Comparison of R²
R²
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
With Anomalies
Without Low Anomalies